System Design: All Concepts + 3 Real Examples

Oct 21, 2026 · @Amar

A complete reference covering every system design concept you need for Google SWE L4 interviews, with three fully worked examples.

Part 1 — Core Concepts

1. Scalability

The ability of a system to handle growing amounts of work by adding resources.

Vertical scaling (scale up): Add more CPU, RAM, or storage to a single machine. Simple but has a physical ceiling.

Horizontal scaling (scale out): Add more machines to the pool. This is what large-scale systems use. Introduces complexity (data consistency, load balancing, statelessness) but is virtually limitless.

Interview signal: Always discuss horizontal scaling for any system expecting millions of users. Mention stateless services, sharding, and load balancers as the enablers.

2. Availability

The percentage of time a system is operational and accessible.

Availability	Downtime/Year
99.9% (three nines)	~8.7 hours
99.99% (four nines)	~52 minutes
99.999% (five nines)	~5.2 minutes

Achieved through redundancy (no single point of failure), failover mechanisms, health checks, and multi-region deployments. In interviews, state your availability target explicitly: "We need 99.99% availability."

3. Reliability

The system produces correct results consistently, even when components fail. Achieved via data replication, checksums, retries with idempotency, and graceful degradation (serve stale data rather than fail entirely).

4. Latency vs Throughput

Latency: Time taken for a single request to travel from client to server and back. Measured in milliseconds.

Throughput: The number of requests a system can handle per unit time. Measured in requests/second.

They are often at odds — batching increases throughput but adds latency. In interviews, state your targets: "P99 latency under 200ms" or "median read latency < 50ms."

5. Consistency Models

Strong consistency: Every read returns the most recent write. Simple for the client, expensive for the system. Example: bank account balance.

Eventual consistency: Reads may return stale data temporarily, but all replicas converge over time. Cheaper and faster. Example: social media feed likes count.

Causal consistency: Operations that are causally related are seen in order; unrelated operations may be seen in any order.

6. CAP Theorem

In a distributed system experiencing a network Partition, you must choose between Consistency and Availability:

- **CP systems** (Consistency + Partition tolerance): Refuse to serve requests if data might be stale. Example: HBase, MongoDB (strong reads).
- **AP systems** (Availability + Partition tolerance): Serve requests even if data might be stale. Example: Cassandra, DynamoDB.

In practice, partitions are rare, so most of the time you get both C and A. The real trade-off is during failures.

7. PACELC Theorem

Extends CAP: if there is a Partition, choose A or C; Else (normal operation), choose Latency or Consistency.

Example: DynamoDB is PA/EL — during partitions it favors availability, and during normal operation it favors low latency (eventual consistency by default).

8. ACID vs BASE

ACID (SQL databases): Atomicity, Consistency, Isolation, Durability. Strong guarantees. Use for financial transactions, inventory management.

BASE (NoSQL databases): Basically Available, Soft state, Eventually consistent. Weaker guarantees but higher performance and scalability. Use for social feeds, analytics, logging.

9. Idempotency

An operation that produces the same result whether executed once or multiple times. Critical for payment systems, message processing, and API retries. Implemented via unique request IDs (idempotency keys).

10. Fault Tolerance

The ability to continue operating when components fail. Techniques: redundancy, retries with exponential backoff, circuit breakers, graceful degradation, health checks, and failover.

Part 2 — Building Blocks

11. DNS (Domain Name System)

Translates domain names to IP addresses. First step in every request. Round-robin DNS provides basic load balancing, and DNS-based routing enables geo-routing for multi-region deployments.

12. Load Balancers

Distribute incoming traffic across multiple servers.

Type	Layer	How It Works	Best For
L4 (Transport)	TCP/UDP	Routes by IP and port	Raw speed, simple routing
L7 (Application)	HTTP	Routes by URL path, headers, cookies	Content-based routing

Algorithms: Round robin, weighted round robin, least connections, IP hash, consistent hashing.

Examples: Nginx, HAProxy, AWS ALB/NLB. Always place a load balancer between clients and API servers.

13. API Gateway

Single entry point for all client requests. Handles authentication, rate limiting, request routing, SSL termination, and protocol translation. In microservices, routes requests to the correct service.

14. Reverse Proxy

Sits in front of servers, handling SSL termination, compression, caching, and load balancing. Often combined with a load balancer. Nginx is the most common example.

15. Databases — SQL vs NoSQL

SQL (relational): PostgreSQL, MySQL. Structured data, ACID transactions, complex queries with JOINs. Best for: user accounts, financial data, anything requiring transactions.

NoSQL types:

Type	Examples	Best For
Key-Value	Redis, DynamoDB	Caching, sessions, simple lookups
Document	MongoDB	User profiles, product catalogs
Wide-Column	Cassandra, HBase	Time-series, messaging, IoT
Graph	Neo4j	Social networks, recommendations

Interview signal: Justify your database choice — "Cassandra for high write throughput in messaging" or "PostgreSQL for ACID guarantees in payments."

16. Database Indexing

Data structures that speed up reads at the cost of slower writes.

- **B-tree index:** Default in PostgreSQL/MySQL. O(log n) reads. Good for range queries.
- **Hash index:** O(1) exact lookups. Cannot do range queries.
- **Composite index:** Index on multiple columns — order matters (leftmost prefix rule).
- **LSM tree:** Used by Cassandra, RocksDB. Optimized for write-heavy workloads.

17. Database Sharding (Partitioning)

Splitting data across multiple database instances.

- **Hash-based:** `shard = hash(key) % num_shards`. Even distribution, but resharding is painful.
- **Range-based:** Shard by ranges (e.g., users A–M on shard 1). Can cause hotspots.
- **Geography-based:** Shard by user region. Good for compliance (GDPR) and latency.
- **Consistent hashing:** Minimizes data movement when adding/removing nodes. Used by DynamoDB, Cassandra.

18. Database Replication

- **Primary-replica (master-slave):** One node handles writes, replicas handle reads. Simple but the primary is a bottleneck.
- **Multi-primary (master-master):** Multiple nodes accept writes. Higher availability but conflict resolution is complex.
- **Sync vs async replication:** Sync guarantees consistency but adds latency. Async is faster but risks data loss on primary failure.

19. Caching

Store frequently accessed data in fast storage (memory) to reduce database load.

Strategy	How It Works	Pros	Cons
Cache-aside	App checks cache first, reads DB on miss	Most common, simple	Cache can go stale
Write-through	Write to cache and DB simultaneously	Always consistent	Slower writes
Write-back	Write to cache first, async flush to DB	Fast writes	Risk of data loss
Read-through	Cache reads from DB on miss	Simpler app logic	Coupling

Eviction policies: LRU (least recently used), LFU (least frequently used), TTL (time-to-live).

Tools: Redis, Memcached. Cache invalidation is one of the two hard problems in CS — always discuss your invalidation strategy.

20. CDN (Content Delivery Network)

Geographically distributed servers that cache static content (images, videos, CSS, JS) close to users.

- **Pull CDN:** Fetches content from origin on first request. Simpler.
- **Push CDN:** You upload content to the CDN explicitly. More control.

Always use for static assets and video streaming. Reduces latency and offloads origin traffic.

21. Message Queues

Async communication between services.

- **Point-to-point (queue):** RabbitMQ. One message goes to one consumer. Good for task distribution.
- **Pub/sub (topic):** Kafka. One message goes to all subscribers. Good for event streaming.

Why use them: Decouple services, handle traffic spikes (buffer), ensure reliability (persist until processed), enable async processing (video transcoding, email).

Kafka is the go-to for high-throughput event streaming.

22. Rate Limiting

Restrict requests a client can make in a time window.

Algorithms: Token bucket, leaky bucket, sliding window, fixed window.

Implementation: Usually at the API gateway layer using Redis for distributed counters.

23. Consistent Hashing

A hashing technique that minimizes redistribution when nodes are added or removed. The hash space is a ring; keys and nodes are mapped onto it. A key is served by the first node clockwise from its position. Virtual nodes ensure even distribution. Used in distributed caches, sharding (DynamoDB, Cassandra), load balancing.

24. Bloom Filters

A probabilistic data structure: "possibly in the set" or "definitely not in the set." False positives possible, false negatives impossible. Very memory-efficient. Used in: checking if a username is taken, web crawlers, database query optimization.

25. Heartbeat & Health Checks

Periodic signals between services to detect failures. If heartbeats stop, the system triggers failover. Load balancers use health checks to stop routing to unhealthy nodes.

Part 3 — Distributed Systems Concepts

26. Consensus Algorithms (Raft, Paxos)

How distributed nodes agree on a single value or leader. Raft is the more understandable one (leader election, log replication, safety). Used in etcd (Kubernetes), CockroachDB. You don't need to implement these, but knowing when to reference them matters — any time you need a distributed lock or leader election.

27. Leader Election

Choosing one node as the coordinator. The leader handles writes or coordination tasks. If the leader fails, a new one is elected. Tools: ZooKeeper, etcd.

28. Distributed Locking

Ensuring mutual exclusion across multiple nodes. Example: only one server should process a payment at a time. Implemented via Redis (Redlock), ZooKeeper, or database advisory locks.

29. Event-Driven Architecture

Services communicate through events rather than direct calls. A service publishes an event ("order placed"); other services react independently (update inventory, send email, charge payment). Enables loose coupling and independent scaling. Kafka and AWS SNS/SQS are common choices.

30. CQRS (Command Query Responsibility Segregation)

Separate the read model from the write model. Writes go to one data store optimized for writes; reads go to another optimized for reads (often denormalized). Common in systems with very different read and write patterns.

31. Event Sourcing

Instead of storing current state, store a log of all state-changing events. Current state is derived by replaying events. Provides a complete audit trail and enables time-travel debugging. Used in financial systems, collaborative editing.

32. Circuit Breaker Pattern

Prevents a service from repeatedly calling a failing downstream service. After a threshold of failures, the circuit "opens" and fails fast (returns an error or cached response). Periodically retries to check if downstream has recovered. Prevents cascade failures.

33. Retry with Exponential Backoff

When a request fails, retry with increasing delays (1s, 2s, 4s, 8s...) plus random jitter. Prevents thundering herd problems where all clients retry simultaneously.

34. Service Discovery

How services find each other in microservices. Client-side: client queries a service registry (Consul, etcd). Server-side: a load balancer queries the registry. Kubernetes provides built-in service discovery via DNS.

35. API Design Best Practices

Protocol	When to Use	Key Feature
REST	External APIs, CRUD operations	Resource-based URLs, HTTP verbs, JSON
gRPC	Internal service-to-service	Binary (Protobuf), multiplexed HTTP/2, low latency
GraphQL	Mobile apps, bandwidth-constrained	Client specifies exact data needed

Always include: cursor-based pagination, versioning, error codes, rate limiting headers.

36. Microservices vs Monolith

Monolith: Single deployable unit. Simpler to develop, test, debug. But becomes unmanageable at scale.

Microservices: Each service owns its domain, data, deployment. Independent scaling, technology diversity, fault isolation. But adds complexity: network latency, distributed transactions, observability.

Interview signal: Don't default to microservices. Mention starting with a modular monolith and splitting services only when there's a scaling reason.

37. Data Serialization

- **JSON:** Human-readable, widely supported. Larger payload.

- **Protocol Buffers (Protobuf):** Binary, strongly typed, smaller, faster. Used with gRPC.
- **Avro:** Schema-embedded, good for Kafka and data pipelines.

38. Observability (Monitoring, Logging, Tracing)

- **Metrics:** Numeric measurements (CPU, latency, error rate). Tools: Prometheus, Datadog.
- **Logging:** Structured event logs. Tools: ELK stack (Elasticsearch, Logstash, Kibana).
- **Distributed tracing:** Track a request across services. Tools: Jaeger, Zipkin.

Mention observability in your designs — it shows production awareness.

39. Object Storage

For storing large unstructured data: images, videos, backups. AWS S3 is the standard. Objects are immutable, addressed by key. Highly durable (11 nines). Always use for media files, never store them in a relational DB.

40. Search Infrastructure

Full-text search over large datasets. Elasticsearch: inverted index, distributed, near-real-time. Use for: search bars, log analysis, filtering products.

41. WebSockets & Long Polling

- **WebSockets:** Full-duplex persistent connection. Server can push data to client any time. Use for: chat, live updates, collaborative editing.
- **Long polling:** Client sends a request; server holds it until there's new data. Simpler but less efficient.
- **Server-Sent Events (SSE):** One-way server-to-client stream. Good for dashboards and live feeds.

42. Data Warehousing & Analytics

OLTP: The main database handling user requests. Optimized for writes. PostgreSQL, MySQL.

OLAP: For heavy analytical queries over large datasets. Snowflake, BigQuery, Redshift.

Separate your analytics workload from your serving database. ETL or streaming pipelines move data from OLTP to OLAP.

43. Geo-Routing & Multi-Region Deployment

Deploy in multiple regions (US, EU, Asia) for lower latency and compliance. DNS-based routing (Route 53) directs users to the nearest region. Challenges: cross-region data replication, conflict resolution, consistency.

Part 4 — Interview Framework (RESHADED)

For every system design interview at Google, follow this structure:

Step	What To Do
R — Requirements	Clarify functional and non-functional requirements. Ask about scale, latency targets, consistency needs.
E — Estimation	Back-of-envelope math: storage, bandwidth, QPS (queries per second).
S — Storage schema	Entity modeling and database choice (SQL vs NoSQL). Design key tables.
H — High-level design	Draw the main components (clients, LB, services, DB, cache, queue). Show data flow.
A — API design	Define core endpoints (REST or gRPC), request/response format.
D — Detailed design	Deep-dive into 2–3 key components. This is where you show depth.
E — Evaluate	Discuss trade-offs, bottlenecks, and failure handling.
D — Distinguish	What makes your design unique? Monitoring, security, cost optimization.

Example 1: Design WhatsApp (Real-Time Messaging)

Requirements

Functional: Send/receive text messages (1-on-1), real-time delivery, read receipts (sent, delivered, read), message history, group chat.

Non-functional: ~10 billion messages/day, message delivery < 100ms for online users, high availability, ~1.5 PB/year storage.

Capacity Estimation

Metric	Value
Average throughput	~115,000 messages/second
Peak throughput	~230,000 messages/second
Message size	~200 bytes
Daily storage	~2 TB/day
Concurrent WebSocket connections	Hundreds of millions

High-Level Architecture

Flow: Mobile App → Load Balancer → API Servers (REST) → Message Queue (Kafka) → Async Workers → Database. Separately, Chat Servers maintain WebSocket connections and push messages to recipients in real-time.

Key Components

1. API Servers (stateless): Handle REST endpoints — send message, fetch history, update read status. Horizontally scalable.

2. Chat Servers (WebSocket): Maintain persistent connections with each online device. When a message arrives, the chat server pushes it to the recipient instantly. Stateful — each server holds connections for a subset of users.

3. Message Distributor (Kafka): Decouples the send path from database writes. API server enqueues the message, returns acknowledgment immediately. Async workers consume from Kafka and persist to DB.

4. Connection Manager (Redis): Tracks which chat server holds each user's WebSocket connection (userId → chatServerId mapping). When user A sends to user B, the system looks up B's chat server and routes there.

5. Offline Queue: If recipient is offline, message is stored and delivered on reconnect. Push notification sent via FCM/APNs.

Database & Sharding

Database: Cassandra — optimized for high write throughput, eventual consistency is acceptable.

Partition key: userId. All of a user's messages live on the same shard. Consistent hashing for even distribution.

Key Trade-offs

- Eventual consistency is fine for messaging (messages arrive in order per conversation, not globally)
- WebSocket servers are stateful — need connection draining on deployment
- End-to-end encryption means the server never reads message content
- Group chats add complexity: fan-out to all members via message queue

Example 2: Design YouTube (Video Streaming Platform)

Requirements

Functional: Upload videos, transcode into multiple resolutions (240p–1080p), stream with adaptive bitrate, search by title/tags/description, like, comment, subscribe.

Non-functional: 10M DAU, 100K uploads/day, 50M views/day, high availability, minimal buffering, no video data loss, cost efficiency.

Capacity Estimation

Metric	Value
Raw video storage	~50 TB/day (100K × 500 MB)
With transcoding (multi-res)	~150 TB/day
Streaming bandwidth	~25 PB/day (before CDN)
Metadata storage	~100 MB/day

High-Level Architecture

The system splits into three paths:

Upload Path: Client → Upload Service → Object Storage (S3) → Message Queue → Transcoding Workers → Transcoded videos stored in S3 → Pushed to CDN.

Streaming Path: Client → CDN edge server (cache hit) → Origin S3 (cache miss). Adaptive bitrate streaming (HLS/DASH) adjusts quality to viewer's bandwidth.

Metadata Path: Client → API Servers → PostgreSQL (metadata), Elasticsearch (search), Redis (view counts, recommendations).

Key Components

1. Upload Service: Accepts video files via chunked upload for large files. Stores raw video in S3. Publishes "video uploaded" event to message queue.

2. Transcoding Pipeline: Workers consume from the queue, transcode each video into multiple resolutions and formats (HLS/DASH segments). CPU-intensive — run on dedicated machines or cloud spot instances.

3. CDN for Video Delivery: Videos served from edge servers close to the viewer. Popular videos cached; long-tail videos fetched from origin on demand (pull CDN). This is where most cost optimization happens.

4. Adaptive Bitrate Streaming (ABR): The player requests chunks in the resolution matching current bandwidth. Server provides a manifest file listing all available qualities.

5. Metadata Service: PostgreSQL for video metadata (title, description, uploader, timestamps). Elasticsearch for full-text search. Redis for fast counters (view count, like count).

6. Recommendation Engine: Based on watch history, likes, collaborative filtering. Runs offline (batch processing) and feeds into a fast-read store.

Database Schema (simplified)

Table	Key Columns
videos	id, title, description, uploader_id, status, created_at
video_segments	video_id, resolution, segment_number, storage_url
users	id, name, email, channel_name
comments	id, video_id, user_id, text, created_at
subscriptions	subscriber_id, channel_id

Key Trade-offs

- Eventual consistency for view/like counts (Redis increment, async flush) — being off by a few seconds is acceptable
- Transcoding is async — delay between upload and video being watchable
- CDN costs are huge — optimize cache hit ratio, use tiered storage (hot/warm/cold)
- Pre-generate thumbnails during transcoding to avoid real-time generation

Example 3: Design a URL Shortener (TinyURL/Bitly)

Requirements

Functional: Create short URL from long URL, redirect short URL to original, optional custom short codes, optional click analytics.

Non-functional: 100M new URLs/day, 100:1 read-to-write ratio, redirects < 50ms, high availability, URLs work forever (or until expiry).

Capacity Estimation

Metric	Value
Write throughput	~1,200/sec
Read throughput	~116,000/sec
Daily storage	~50 GB/day
5-year storage	~91 TB
Short code space (7 chars, Base62)	~3.5 trillion combinations
Cache needed	~100 GB (fits in Redis cluster)

API Design

- `POST /shorten` — body: {long_url, custom_code?, expiry?} → returns short_url
- `GET /{short_code}` → 302 redirect to long_url

High-Level Architecture

Create flow: Client → Load Balancer / API Gateway → Shortening Service → Key Generation Service (KGS) provides a pre-generated code → Store mapping in Database.

Redirect flow: Client → Load Balancer → Redirect Service → Redis Cache (hit) or Database (miss) → 302 Redirect. Each redirect also publishes a click event to Kafka → Analytics Service → ClickHouse.

Key Components

1. Key Generation Service (KGS) — recommended approach: Pre-generates a pool of random 7-character Base62 codes. When the shortening service needs a code, it atomically takes one from the pool. Avoids collision handling entirely.

Alternative approaches:

Approach	Pros	Cons
KGS (pre-generated pool)	No collisions, unguessable	Pool management needed
Hash + truncate (MD5/SHA)	Simple	Collisions need handling
Auto-increment → Base62	Collision-free	Sequential (guessable), SPOF

2. Redirect Service: Checks Redis cache first (cache-aside), falls back to database. Returns a 302 redirect.

3. 301 vs 302 Redirect:

- **301 (permanent):** Browser caches it. Reduces server load but can't track clicks or change destination.
- **302 (temporary):** Every click hits your server. Enables analytics and destination changes. Bitly uses 302.

4. Analytics Pipeline: Each redirect publishes a click event to Kafka (async, doesn't block redirect). Flink or Spark Streaming aggregates by time, country, referrer into ClickHouse or Druid. Redis INCR for real-time counters.

5. Database: Sharded PostgreSQL by short_code (each redirect hits one shard) or DynamoDB for simple key-value lookups.

Schema: short_code (PK), long_url, user_id, created_at, expires_at.

6. Caching: Redis with 24-hour TTL. Targets 90%+ cache hit rate. Cache-aside pattern.

Key Trade-offs

- KGS vs hash-based generation (pool management vs collision handling)
- 301 vs 302 (server load vs analytics)
- SQL vs NoSQL (transaction guarantees vs key-value simplicity)
- Sync counters vs async event streaming (accuracy vs performance)

Quick Reference: Top 10 Google System Design Problems

#	Problem	Key Concepts Tested
1	Design Google Search	Web crawling, inverted index, PageRank, caching

#	Problem	Key Concepts Tested
2	Design YouTube	Video transcoding, CDN, adaptive streaming, metadata
3	Design Google Maps	Geospatial indexing (QuadTree), routing algorithms, tile serving
4	Design Google Drive	File sync, chunking, deduplication, conflict resolution
5	Design Gmail	Email protocols (SMTP/IMAP), search indexing, spam filtering
6	Design WhatsApp/Chat	WebSockets, message queues, presence, E2E encryption
7	Design URL Shortener	Hashing, key generation, caching, analytics pipeline
8	Design Twitter/Feed	Fan-out on write vs read, ranking, caching, pub/sub
9	Design Uber/Lyft	Geo-hashing, matching algorithm, real-time tracking, surge pricing
10	Design Rate Limiter	Token bucket, sliding window, distributed counters

Sources

- The 2026 System Design Interview Cheat Sheet — DesignGurus
- System Design Interview: Design WhatsApp — Level Up Coding
- Design YouTube — AlgoMaster
- URL Shortener System Design — TechInterview
- Expert Tips for Google System Design — DesignGurus